

PDF

Academic Documentation

تحليل — OpenClaw Manager شامل للنظام

System Analysis Document \ SaaS Product
Architecture

Ahmed \ Team

Layout

A4 printable, colored cover, cards,
tables, and code blocks

Document Type

Academic / Technical Report

Generation

HTML to PDF using local
Chromium/Puppeteer

Typography

Arabic-friendly fonts with mixed
RTL/LTR support

OpenClaw Manager – تحليل شامل للنظام

< **System Analysis Document** > وثيقة تحليل نظام متكامل لمنتج إدارة OpenClaw للمستخدم العادي عبر تطبيق Windows (Tauri)

جدول المحتويات

- [مقدمة وملخص تنفيذي] (#1-مقدمة-وملخص-تنفيذي)
- [هدف المشروع] (#2-هدف-المشروع)
- [الجمهور المستهدف] (#3-الجمهور-المستهدف)
- [المعمارية العامة للنظام] (#4-المعمارية-العامة-للنظام)
- [المكونات بالتفصيل] (#5-المكونات-بالتفصيل)
- [سير العمل والتدفقات] (#6-سير-العمل-والتدفقات)
- [Firebase – الهيكل والتكامل] (#7-firebase-الهيكل-والتكامل)
- [نظام الـ AI Error Recovery] (#8-نظام-ال-ai-error-recovery)
- [الأمن والخصوصية] (#9-الأمن-والخصوصية)
- [خريطة الطريق (Roadmap)] (#10-خريطة-الطريق-roadmap)
- [المخاطر والتخفيف] (#11-المخاطر-والتخفيف)

1. مقدمة وملخص تنفيذي

1.1 نظرة عامة

OpenClaw هو مساعد شخصي مفتوح المصدر ذكي، لكنه يتطلب تنصيبًا يدويًا وإدارة تقنية متقدمة. عندما يواجه المستخدم العادي مشكلة، يضطر للعودة إلى المطور. هذا يُنشئ عنق زجاجة ويمنع التوسع.

OpenClaw Manager هو منتج SaaS يبني طبقة إدارة ذكية فوق OpenClaw، تسمح لأي مستخدم عادي باستخدام OpenClaw دون الحاجة إلى خبرة تقنية، مع نظام ذكاء اصطناعي مدمج يشخص ويحل المشاكل تلقائيًا.

1.2 الفروقات الرئيسية عن OpenClaw الخام

| الميزة | OpenClaw الخام | OpenClaw Manager | |-----|-----|-----| | التنصيب | سطر أوامر يدوي | تطبيق Windows بنقرة واحدة | | الإدارة | عبر CLI/Control UI (تقنية) | واجهة مبسطة | | حل المشاكل | يحتاج مطور AI | يصلح تلقائيًا | | تتبع الاستهلاك | يدوي | لوحة تحكم + Firestore | | إدارة المستخدمين | لا يوجد | + Firebase Auth | | Admin Panel | الأجهزة | مفتوح | جهاز واحد لكل مستخدم |

1.3 البيان المعماري (Problem Statement)

< **للاستفادة من OpenClaw، يحتاج المستخدم إلى تثبيت يدوي، إدارة عبر CLI، وعند أي عطل يجب أن يعود للمطور. منتجنا يبني طبقة SaaS فوق OpenClaw تسمح لأي مستخدم عادي بنصبه واستخدامه وإدارته، مع AI agent يكتشف ويشخص ويحل %90+ من المشاكل تلقائيًا. **

2. هدف المشروع

2.1 الأهداف الرئيسية

- **GOAL-1** إنشاء تطبيق Windows أصلي (Tauri) يثبت ويدير OpenClaw
- **GOAL-2** توفير لوحة تحكم مرئية لحالة المساعد والقنوات والموديلات
- **GOAL-3** بناء AI Error Agent يشخص ويصلح مشاكل OpenClaw تلقائيًا
- **GOAL-4** نظام SaaS مع Firebase Auth يتتبع المستخدمين والاشتراكات
- **GOAL-5** ربط جهاز واحد لكل مستخدم مع إمكانية تعطيل الحساب من بعد

2.2 مؤشرات النجاح (KPIs)

- وقت تثبيت OpenClaw للمستخدم العادي: > 5 دقائق
- نسبة المشاكل التي يحلها > 85% AI Error Agent
- وقت الاستجابة للمشاكل: < دقيقة (تلقائي)
- عدد الزيارات للمطور بخصوص مشاكل التشغيل: صفر

3. الجمهور المستهدف

3.1 الشخصيات المستهدفة

- **المستخدم العادي (Non-Technical User)**:
- ليس لديه خبرة في CLI/Linux
- يريد مساعد شخصي يعمل فور تثبيته
- لا يعرف ما هو WSL
- يتوقع تجربة "Install & Forget"
- **المستخدم شبه التقني (Prosumer)**:
- يعرف أساسيات الكمبيوتر
- قد يكون جرب ChatGPT/web
- يريد تحكمًا بسيطًا في الموديلات
- مهتم بالتكلفة والاستهلاك

3.2 متطلبات المستخدم

- واجهة بالعربية (أساسي) والإنجليزية
- أزرار كبيرة وواضحة — لا أوامر نصية
- تحديث الحالة فوري (Online/Offline/Error)
- تنبيهات عند حدوث مشكلة
- زر واحد لحل المشكلة "Fix"

4. المعمارية العامة للنظام





4.2 مبادئ التصميم (Design Principles)

- **Fail-Safe Pyramid**: كل طبقة لها fallback للطبقة التي تحتها
- **Self-Healing by Default**: النظام يحاول الإصلاح قبل إزعاج المستخدم
- **Single Device Binding**: حساب واحد = جهاز واحد
- **Offline-First**: التطبيق يعمل حتى لو انقطع الإنترنت (للمهام المحلية)
- **Observable Always**: المستخدم يرى حالة كل شيء في أي وقت

5. المكونات بالتفصيل

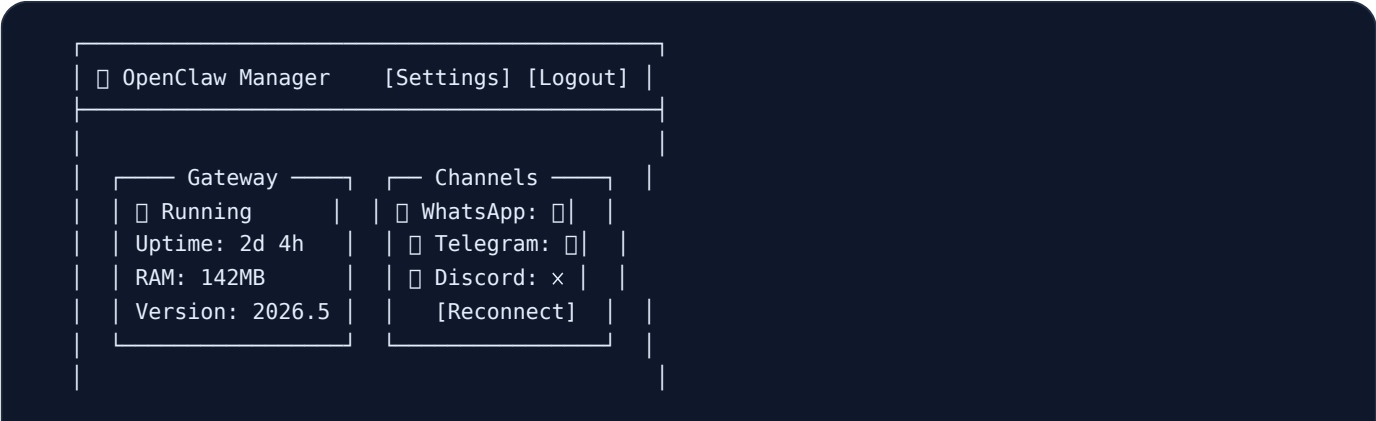
5.1 Windows Application (Tauri)

5.1.1 التقنيات

التقنية | الدور | **Tauri v2** | إطار تطبيق سطح المكتب (Rust + Web UI) | **React / Lit** | واجهة المستخدم | **serde + serde_json** | business logic, WSL bridge, Firebase SDK | **Rust** | تحليل JSON من **tauri-plugin-shell** | **WSL/OpenClaw** | تشغيل أوامر HTTP calls | **reqwest** | **WSL/shell** | **Firebase REST** | API

5.1.2 نوافذ التطبيق

- **Dashboard** (الصفحة الرئيسية)



```
┌── Model Usage ──┐ ┌── Sessions ──┐
│ Sonnet: 1.2K tks │ │ Active: 3    │
│ GPT: 800 tks    │ │ Today: 47 msgs │
│ GPT Cost: $0.24 │ └──────────┘
└────────────────┘

[ ] AI Error Assistant [ ] Full Logs
```

****AI Error Assistant**** •

```
[ ] AI Error Assistant
```

```
[ ] System Check:
[ ] Gateway reachable
x WhatsApp disconnected
[ ] Telegram connected
...
```

```
[ ] Diagnosis:
"WhatsApp جلسة الأمان انتهت..."
```

```
[ ] Fix: Reconnect WhatsApp]
```

```
┌── Chat History ──┐
│ [12:00] Fix: تم إعادة الربط │
│ [11:45] Error: WhatsApp off │
└────────────────┘
```

****Channels Manager**** •

```
[ ] Channel Configuration
```

```
┌── WhatsApp ──┐
│ Status: [ ] Connected │
│ Phone: +9665xxxxxxx   │
│ [QR Login] [Disconnect] │
└──────────┘
```

```
┌── Telegram ──┐
│ Status: [ ] Connected │
│ Bot: @MyAssistantBot  │
│ [Change Token] [Disconnect] │
└──────────┘
```

```
[ ] Add Channel]
```

****Model Monitor**** •

حالة WSL:

- | Running → متابعة
- | Stopped → wsl -d Ubuntu
- | Installing → مؤشر تثبيت WSL
- | Error → إعادة تعيين WSL

حالة Gateway:

- | Reachable عبر WS → اتصال مباشر
- | Process not running → restart gateway
- | Doctor needed → run doctor --fix
- | Fatal → restart WSL

OpenClaw Gateway Integration 5.3

التطبيق يتصل مع Gateway بطريقتين:

الطريقة الأولى: WebSocket RPC (المفضلة)

```
Tauri → ws://127.0.0.1:18789 → connect → chat.send → agent maintenance
```

Response events: agent events with tool execution results

- يرسل أوامر مباشرة لـ `agent` maintenance
- يستقبل تحديثات فورية (Presence, Health, Session events)
- يوفر chat UI للـ AI Error Assistant

الطريقة الثانية: WSL CLI (Fallback)

```
Tauri → wsl -d Ubuntu -- openclaw health --json  
→ wsl -d Ubuntu -- openclaw config get agents
```

- يستخدم عندما Gateway بايظ
- أبطأ ويحتاج parse النتائج
- لكنه يعمل دائماً

5.3.1 هيكل الـ Maintenance Agent

```
// openclaw.json تضاف إلى  
{  
  agents: {  
    list: [{  
      id: "maintenance",  
      name: "مساعد الصيانة",  
      model: {  
        primary: "anthropic/claude-sonnet-4-6",  
      },  
      tools: {  
        scriptbox: { enabled: true }, // آمنة لـ scripts لتشغيل  
      },  
      skills: ["maintenance-skills"],  
      prompt: `أنت مساعد صيانة OpenClaw.  
مهمتك تشخيص وحل مشاكل النظام`  
    }  
  }  
}
```


لديك الأدوات التالية:

- run_health_check: يجري openclaw health --json
- run_doctor: يجري openclaw doctor --fix
- read_gateway_logs: logs يقرأ آخر
- restart_gateway: Gateway يعيد تشغيل
- reconnect_channel(channel): يعيد ربط قناة
- get_config(path): يقرأ إعدادات
- run_diagnostics: يصدر تقرير تشخيص

قاعدة التشخيص:

1. أولاً health افحص
2. logs إذا فشل → افحص
3. doctor --fix إذا وجدت خطأ → استخدم
4. config إذا استمر → افحص
5. restart gateway → إذا فشل كل شيء
6. `وابلع المستخدم diagnostics إذا فشل → اصدر`
}],
,
},
}

Error Recovery Orchestrator 5.4

Fallback Pyramid 5.4.1

```
□ LEVEL 1: Gateway RPC
OpenClaw Gateway يعمل ← chat.send → maintenance agent
|
| Gateway disconnected
▼
□ LEVEL 2: WSL CLI
WSL يعمل ← wsl openclaw health, wsl openclaw doctor
|
| Gateway won't start
▼
□ LEVEL 3: WSL Reset
← wsl --shutdown → wsl → إعادة تشغيل OpenClaw
|
| WSL corrupt
▼
□ LEVEL 4: PowerShell Recovery
← إعادة تثبيت WSL ← إعادة تثبيت OpenClaw
|
| Everything failed
▼
□ ESCALATION: Developer Alert
← إشعار للمطور ← diagnostics تصدير
```

Playbooks 5.4.2 الجاهزة

****Playbook: WhatsApp Disconnected****

1. Detect: health check → channel=whatsapp, status=failure
2. Diagnose: read logs → "WhatsApp session expired"
3. Fix strategy:
 - a. Run: wsl openclaw channels logout --channel whatsapp
 - b. Open QR modal in Tauri
 - c. Run: wsl openclaw channels login --channel whatsapp

- d. Wait for QR, display in UI
4. Verify: re-run health check
5. Report: "QR تم إعادة ربط واتساب، يرجى مسح"

****Playbook: Gateway Not Responding****

1. Detect: WS connect timeout
2. Diagnose: wsl openclaw gateway status
 - If "not running" → start
 - If "crashed" → check logs → doctor
3. Fix:
 - a. wsl openclaw doctor --fix --non-interactive
 - b. wsl openclaw gateway restart
 - c. Wait 10s, retry WS connection
4. Verify: openclaw health --json
5. If still down → level 3 (WSL reset)

****Playbook: Out of Memory / High Resource Usage****

1. Detect: health check → memory > 500MB
2. Diagnose: log analysis
3. Fix:
 - a. Check which sessions are active
 - b. [Optional] Restart gateway
 - c. Apply resource limits if configured
4. Verify: re-check memory

6. سير العمل والتدفقات

6.1 التدفق الرئيسي للمستخدم





6.2 تدفق AI Error Recovery



```
Apply Fix
- Execute playbook steps
- Wait for each step result
- Retry if needed (max 3x)
```

```
Verify & Report
- Re-run health check
- ☑ Success → إشعار المستخدم
- ✕ Failed → Escalate
```

6.3 تدفق SaaS Auth

```
Tauri App: First Launch
```

```
Firebase Auth: Sign Up
email + password
Create user in Firestore
subscription: trial
device_count: 0
```

```
Device Registration
device_fingerprint ← HW_ID
POST /api/devices/register
Check: user.devices < 1
Check: device not bound to other
Save: user.devices[0] = fingerprint
```

```
Admin API Check (عند كل تشغيل)
GET /api/users/{uid}/status
← active → متابعة
← disabled → "إظهار حسابك موقوف"
```

6.4 إلغاء ربط الجهاز (Device Transfer)

الجهاز القديم:

1. المستخدم يضغط "إلغاء ربط الجهاز"
2. من Firestore device_fingerprint التطبيق يسمح
3. تبقى بيانات المستخدم محفوظة

الجهاز الجديد:

1. تسجيل الدخول بنفس الحساب
2. التطبيق يكتشف no device_bound
3. device_count=0 يشوف
4. يربط الجهاز الجديد

، ملاحظة: لو المستخدم فقد الجهاز
يقدر بمسح الجهاز Admin Panel المطور من
، ويدع المستخدم يربط واحد جديد

7. Firebase – الهيكل والتكامل

7.1 هيكل Firestore

```
/users/{uid}
├─ email: string
├─ displayName: string
├─ createdAt: timestamp
├─ lastActive: timestamp
├─ status: "active" | "disabled" | "suspended"
├─ subscription:
│   └─ plan: "trial" | "basic" | "pro"
│   └─ startDate: timestamp
│   └─ endDate: timestamp (null = active)
│   └─ stripeCustomerId: string (للمستقبل)
├─ device:
│   └─ fingerprint: string
│   └─ name: string (optional)
│   └─ os: string
│   └─ boundAt: timestamp
│   └─ lastSeen: timestamp
├─ settings:
│   └─ language: "ar" | "en"
│   └─ autoUpdate: boolean
│   └─ notifications: boolean
└─ usage:
    └─ todayTokens: number
    └─ monthTokens: number
    └─ todayCost: number
    └─ monthCost: number
    └─ lastReset: timestamp

/devices/{fingerprint}
├─ boundTo: uid (null if unbound)
├─ firstSeen: timestamp
├─ lastSeen: timestamp
└─ hwInfo: string (non-identifying)

/admin/users (Admin Panel فقط)
├─ list: [uid, email, status, device?]
└─ stats:
    └─ totalUsers: number
    └─ activeUsers: number
    └─ disabledUsers: number
```

7.2 Firebase Functions (Cloud)

تنشئ وثيقة	Function	Trigger	Description
Auth onCreate	`onUserCreate`	Firestore	المستخدم في Auth onDelete
المستخدم في Auth onDelete	`onUserDelete`	Firestore	تنظف بيانات المستخدم
Cron (daily)	resetUsage	Cron (daily)	تعيد ضبط counters اليومية
Cron (daily)			تفحص انتهاء الاشتراكات

المستخدم | adminDisableUser` | Callable` | توقف حساب مستخدم | | Callable` | adminResetDevice` | تمسح ربط جهاز

Firestore Security Rules 7.3

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // المستخدم يقرأ بياناته
    match /users/{userId} {
      allow read, write: if request.auth.uid == userId
        || request.auth.token.admin == true;
    }

    // الأجهزة - للقراءة فقط للمستخدم
    match /devices/{fingerprint} {
      allow read: if resource.data.boundTo == request.auth.uid;
      allow create: if request.auth.uid != null;
    }

    // Admin فقط
    match /admin/{document} {
      allow read, write: if request.auth.token.admin == true;
    }
  }
}
```

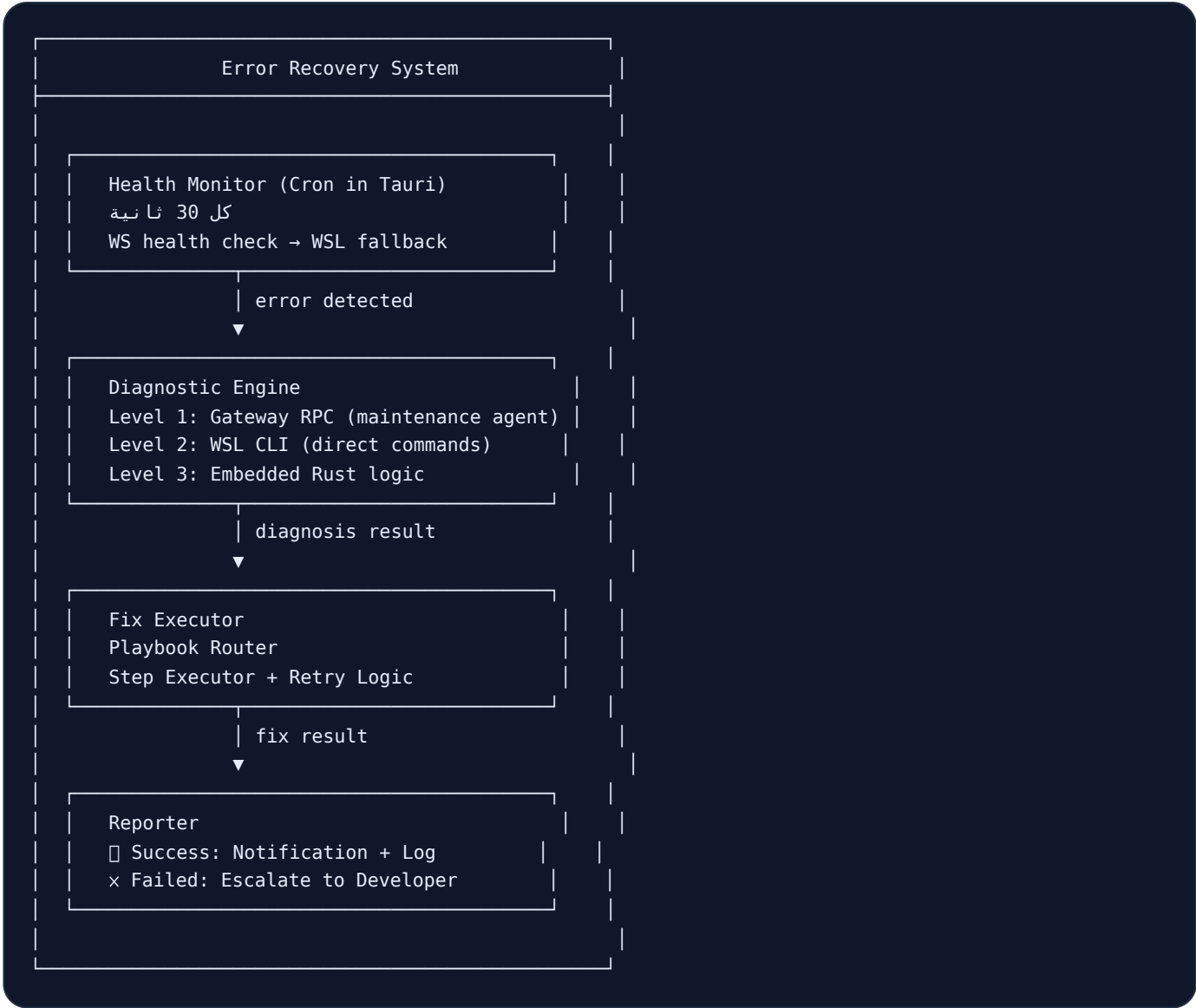
7.4 تدفق التحقق من الجهاز

1. Tauri يرسل:

```
{
  uid: "abc123",
  email: "user@email.com",
  deviceFingerprint: "sha256(hw_id+os+disk_id)"
}
```
2. Firebase Function:
 - a. Get user doc → check status
 - b. If disabled → return {error: "disabled"}
 - c. Get device doc by fingerprint
 - d. If device.boundTo == null → bind to this user
 - e. If device.boundTo != uid → return {error: "device_bound"}
 - f. If device.boundTo == uid → allow
3. Tauri:
 - If allowed → continue to dashboard
 - If disabled → show "حسابك موقوف"
 - If device_bound → show "هذا الجهاز مرتبط بحساب آخر"

8. نظام الـ AI Error Recovery

8.1 هندسة النظام



Known Error Patterns Playbook 8.2

الخطأ | التشخيص | الحل | نسبة النجاح | |-----|-----|-----|-----| | WhatsApp expired | Log: "loggedOut" | | 100% | | Re-run QR login | 100% | | Gateway crash | WS timeout + status check | doctor + restart | 90% | | Config key | 100% | | doctor --fix | 85% | | API key expired | Model calls fail | UI | invalid | doctor | | Port conflict | "EADDRINUSE" | Stop other process + restart | 80% | | WSL stopped | wsl --status | wsl restart | 100% | | High memory | memory > 500MB | Restart gateway | 70% | | SSL/Proxy errors | Network | — | Diagnose + Escalate | لا يتطابق مع الأنماط | errors | Diagnose + report | 60% | | Unknown error

8.3 التعامل مع Unknown Errors

عندما لا يتعرف النظام على الخطأ:

- تشغيل فحص شامل:
 - `openclaw health --verbose --json`
 - `openclaw doctor --non-interactive`
 - logs سطر آخر من 100
- Developer Dashboard إرسال البيانات إلى:
 - Diagnostics export
 - Error context + logs
- إظهار رسالة للمستخدم:

عذرًا، واجه النظام مشكلة غير معروفة"
".تم إرسال التقرير للمطور. سنتواصل معك قريبًا"

4. النظام يستمر في المراقبة :

- إذا تحسنت الحالة → إشعار -
- إذا تفاقمت → إعادة تشغيل تلقائي -

(الأُسبوعان 3-4) Phase 2 — Control & Diagnostics

التحكم والتشخيص: Week 3-4

- Channels Manager UI
 - WhatsApp QR login
 - Telegram bot config
- Log viewer with filtering
- OpenClaw Maintenance Agent config
- WebSocket integration (Tauri ↔ Gateway)
- Error Recovery Orchestrator (Level 1 & 2)
 - Auto-detect disconnects
 - Auto-fix playbooks
- Model Monitor (basic: tokens + cost)

Phase 3 — Intelligence & Production (الأسبوع 5-6)

الذكاء والإنتاج: Week 5-6

- AI Error Assistant Chat UI
- Level 3 Recovery (WSL reset)
- Developer Admin Panel (React)
 - User list
 - Disable/enable accounts
 - Device unbinding
 - Usage overview
- Usage reporting to Firestore
- Model switching from UI
- Notification system
- Beta testing & bug fixes

Phase 4 — Post-Launch (الأسبوع +7)

ما بعد الإطلاق: Week 7+

- Auto-update mechanism
- Subscription/Pricing (Stripe)
- More error playbooks
- Performance optimization
- Multi-language support
- Public installer signing
- User documentation

11. المخاطر والتخفيف

11.1 المخاطر التقنية

Abstract | الخطر | الاحتمال | التأثير | التخفيف | |-----|-----|-----|-----|
| layer فوق WSL | API calls غير مستقر | منخفض | عالي | Firebase pricing | | Fallback to alternative (Docker) |
متوسط | متوسط | Cosmos أو Supabase كبديل | | Device fingerprint غير دقيق | منخفض | متوسط | | Fingerprint +
| Tauri v2 bugs | fallback manual confirm | منخفض | متوسط | | Test early; keep fallbacks

11.2 مخاطر الأعمال

Admin | متوسط | متوسط | المستخدم يفقد جهازه | |-----|-----|-----|-----| | التخفيف | التأثير | الاحتمال | الخطر |
override | لربط جهاز جديد | | المستخدم يبطل WSL | منخفض | عالي | Installer يسوي API key | | repair مسرية | منخفض |
عالي | Rotation mechanism + alerts | | عدم رضا عن الـ AI agent | متوسط | منخفض | Manual override متاح دائمًا |

11.3 استراتيجية الخروج (Fallback Plan)

إذا فشل المشروع لأي سبب

1. Firebase/SaaS layer إيقاف
2. Offline-only mode يتحول التطبيق إلى
3. OpenClaw + WSL المستخدم يحتفظ بـ
4. التطبيق يصبح مجرد إدارة محلية
5. المفتوح OpenClaw توجيه المستخدمين لمجتمع

الملحق أ: المصطلحات

المصطلح | الشرح | |-----|-----| |**Gateway**| خدمة OpenClaw الرئيسية (WebSocket + HTTP) |**WSL** |**|| (Windows Subsystem for Linux — بيئة Linux داخل **Agent** | |**Windows**| وكيل ذكي داخل **Tool** | |**OpenClaw**| دالة يستخدمها ال agent (مثل run_doctor) | |**Channel**| قناة تواصل (WhatsApp, Telegram, إلخ) | |**Tauri**| إطار تطبيقات سطح المكتب (Rust + Web) | |**Playbook**| سلسلة خطوات لحل مشكلة معينة |

الملحق ب: هيكل مجلدات المشروع

```

└─ openclaw-manager/
    └─ src-tauri/
        # Rust backend
        └─ src/
            └─ main.rs
                # Entry point
            └─ wsl_bridge.rs
                # WSL command execution
            └─ oc_ws_client.rs
                # OpenClaw WebSocket client
            └─ firebase.rs
                # Firebase Auth + Firestore
            └─ orchestrator.rs
                # Error recovery engine
            └─ diagnostics.rs
                # Diagnostic utilities
            └─ device.rs
                # Device fingerprint
        └─ Cargo.toml
        └─ tauri.conf.json

└─ src/
    # Frontend (React/Lit)
    └─ App.tsx
    └─ pages/
        └─ Dashboard.tsx
        └─ Channels.tsx
        └─ ModelMonitor.tsx
        └─ AIAssistant.tsx
        └─ Logs.tsx
        └─ Settings.tsx
    └─ components/
        └─ StatusCard.tsx
        └─ ChannelRow.tsx
        └─ Navigation.tsx
    └─ styles/

└─ installer/

```

```
| |─ setup.iss          # Inno Setup (optional)
| |─ wsl_install.ps1   # PowerShell bootstrap
| |─ openclaw_install.sh # Bash script for WSL
|
|─ admin-panel/        # React Admin Dashboard
| |─ src/
| | |─ pages/
| | | |─ Users.tsx
| | | |─ Analytics.tsx
| | |─ firebase-admin.ts
| |─ ...
|
|─ oclaw-config/
| |─ maintenance-agent.json # Agent config template
| |─ openclaw.base.json    # Base config for new users
|
|─ functions/          # Firebase Cloud Functions
| |─ src/
| | |─ onUserCreate.ts
| | |─ onUserDelete.ts
| | |─ adminDisableUser.ts
| | |─ checkSubscription.ts
| |─ package.json
|
|─ package.json        # Tauri frontend deps
```

Status:**** < (Assistant محمد) Document Version:** 1.0 > **Date:** 2026-05-15 > **Author:** Ahmed** <
Draft for Review